Phrase01

我一开始按照 lab 中给出的
objdump -d --no-show-raw-insn --disassemble=phase_01 ./bomb > phase_01.S
获取了 phase_01 的汇编，，并使用了 dsh 帮助我阅读这些汇编代码（标黄是我认为的关键汇编，蓝色为理解，后面同上）：

```
402020: endbr64                          ; CET 间接跳转目标标记，不影响数据
402024: push   %rbp
402025: mov    %rsp,%rbp                 ; 建立栈帧
402028: push   %r12                      ; r12 是 callee-saved，跨调用保存
40202a: mov    %rsi,%r12                 ; r12 = 第二个参数 challenge
40202d: mov    $0x1,%esi                 ; valid_u32_fields 的第 2 个参数 = 字段个数 1
402032: push   %rbx
402033: mov    %rdi,%rbx                 ; rbx = 第一个参数 input 字符串
402036: sub    $0x10,%rsp                ; 开 16 字节局部空间
40203a: call   402c00 <valid_u32_fields>
40203f: test   %al,%al                   ; 返回值是 bool，放在 al
402041: jne    402050                    ; 校验通过才往下走
402043: add $0x10,%rsp                   ; 失败路径：恢复栈
402047: xor %eax,%eax                    ; 返回 false
402049: pop %rbx / pop %r12 / pop %rbp / ret
40204e: xchg %ax,%ax                     ; 2 字节 nop，用于对齐
402050: xor    %eax,%eax                 ; al = 0：sscanf 是变参函数，SysV ABI 要求 al 存放
                                         ;用的 SSE 寄存器个数，这里没有浮点参数，置 0
402052: lea    -0x14(%rbp),%rdx          ; 第 3 个参数 = 输出变量的地址（4 字节，u32）
402056: mov    $0x403182,%esi            ; 第 2 个参数 = .rodata 里的格式串（大概率 "%u"）
40205b: mov    %rbx,%rdi                 ; 第 1 个参数 = 输入串
40205e: call   __isoc99_sscanf@plt
402063: cmp    $0x1,%eax                 ; 必须恰好成功转换 1 项
402066: jne    402043                    ; 否则失败
402068: mov    %r12,%rdi                 ;
调 p1_compute 时把 challenge 放进 rdi（第 1 个参数）
40206b: call   401fe0 <p1_compute>
402070: cmp    %eax,-0x14(%rbp)          ;
AT&T 语法方向：内存值 - eax，即“输入 - 返回值”
402073: sete   %al                       ; 相等 → al = 1 → 返回 true
402076: add $0x10,%rsp / pop... / ret
```

我发现这里调用了函数 p1_compute 和 valid_u32_fields，我在 phrase_support.c 里找到了 valid_u32_fields，但是其他文件中我没有找到 p1_compute，于是求助于 dsh 让他帮我找这一部分的源码，最后在 bomb 文件中找到了，因为这个文件是由二进制编写的，我没有安装对应的阅读插件，因此我一开始只看到了乱码，就没有找到 p1_compute。

找到 p1_compute 这个函数之后，我让 dsh 帮我 dump 出来并解释了这个函数的源码：

```
401fe0: endbr64
401fe4: push %rbp
401fe5: mov    %rdi,%rax                ; rax = challenge（复制三份备用）
401fe8: mov    %rdi,%rsi
401feb: mov    %rdi,%rdx
401fee: shr    $0x20,%rax               ; rax = challenge >> 32   → 高 32 位
401ff2: shr    $0x10,%rsi               ; rsi = challenge >> 16   → 低 32 位是 bits 16..47
401ff6: mov    %eax,%r8d                ; r8d = hi（暂存，稍后当实参）
401ff9: mov    %eax,%edi                ; edi = hi
401ffb: mov    %rsp,%rbp                ; 无数据效果
401ffe: call 401fc0 <p1_step1>          ; step1(edi=hi, esi=(challenge>>16)的低 32 位)
402003: mov    %r8d,%esi                ; esi = hi
402006: mov    %edx,%edi                ; edi = challenge 的低 32 位（rdx 从未移位）
402008: mov    %eax,%ecx                ; ecx = step1 的返回值（eax 马上要被覆盖，先存起来）
40200a: call 401fb0 <p1_step0>          ; step0(edi=lo, esi=hi)
40200f: mov    %ecx,%esi                ; esi = step1 的结果
402011: mov    %eax,%edi                ; edi = step0 的结果
402013: call 401fd0 <p1_step2>          ; step2(step0 结果, step1 结果)
402018: pop    %rbp
402019: ret                             ; 返回值在 eax（32 位）
```

阅读后我发现其中又调用了 p1_step0，p1_step1，p1_step2 三个函数，因此我再次让 dsh 帮我 dump 出这几个小函数并解释：

```
p1_step0:
401fb4: lea 0x1840(%rsi),%eax     ; eax = esi + 0x1840（32 位加法，溢出回绕）
401fba: xor %edi,%eax             ; eax ^= edi

p1_step1:
401fc4: sub %esi,%edi                       ; edi = edi - esi（32 位减法）
401fc6: lea 0xc3b6(%rdi,%rdi,4),%eax        ; eax = rdi + rdi*4 + 0xc3b6 = 5*rdi + 0xc3b6

p1_step2:
401fd4: xor $0xf276,%esi                    ; esi ^= 0xf276
401fda: lea (%rsi,%rdi,8),%eax              ; eax = rdi*8 + rsi
```

至此，第一题的信息收集完毕，可见为了解决这道题目需要先吧 p1_step0，p1_step1，p1_step2 从源码转换为函数，再在 p1_compute 中调用这些函数，实现 p1_compute 的功能，最后实现 phrase_01 的算法。

一下是是我根据 p1_step0，p1_step1，p1_step2 的汇编代码实现的三个函数：
这三个辅助函数的意义比较清楚，全都是功能性的代码，不涉及输入校验，变参调用准备，失败清理路径等等，依据汇编语言代码的意义直接写出功能即可

```
static uint32_t p1_step0(uint32_t a, uint32_t b) {
    return (b + 0x1840u) ^ a;
}

static uint32_t p1_step1(uint32_t a, uint32_t b) {
    return (a - b) * 5u + 0xc3b6u;
}

static uint32_t p1_step2(uint32_t a, uint32_t b) {
    return a * 8u + (b ^ 0xf276u);
}
```

再根据 p1_compute 的汇编代码，调用 p1_step0，p1_step1，p1_step2，实现 p1_compute 的功能：
由 p1_compute 的汇编代码，我们能确定三个量

lo	(uint32_t)challenge
hi	(uint32_t)(challenge >> 32)
mid	(uint32_t)(challenge >> 16)

再由 mov 语句确定这三个量被调用的顺序
1. p1_step1(hi, mid)： %edi=hi、%esi=(challenge>>16) 的低 32 位
2. p1_step0(lo, hi)： %edi=lo、%esi=%r8d（=hi）
3. p1_step2(step0 结果, step1 结果)：%edi=step0 的 %eax、%esi=%ecx（step1 的结果）
这样我们就可以实现 p1_comput 的功能

```
static uint32_t p1_compute(uint64_t challenge) {
    uint32_t lo  = (uint32_t)challenge;
    uint32_t hi  = (uint32_t)(challenge >> 32);
    uint32_t mid = (uint32_t)(challenge >> 16);
    uint32_t t1 = p1_step1(hi, mid);
    uint32_t t0 = p1_step0(lo, hi);
    return p1_step2(t0, t1);
}
```

最后根据 phrase_01 的汇编代码，调用 p1_compute，完成整个算法：
mov $0x1,%esi 说明只允许 1 个 u32 字段
cmp $0x1,%eax 要求 sscanf 恰好转换出 1 项，因此输入是一个 uint32_t
402068 把 phase_01 的第二个参数 (challenge, 保存在 %r12) 放进 %rdi 再调用 p1_compute，所以返回值只依赖 challenge

cmp %eax,-0x14(%rbp) + sete 是 32 位相等比较
因此一个十进制 u32，其值等于 p1_compute(challenge) 的 32 位返回值
其余汇编代码都在做输入校验，变参调用等准备工作，均由编译器自己完成

```
int solve_phase_1(uint64_t challenge, char *out, size_t cap) {
    uint32_t values[MAX_FIELDS] = {0};
    size_t count = 1;
    values[0] = p1_compute(challenge);
    return write_answer(out, cap, values, count);
}
```

Seed 验证：





Agent 用途：帮助理解源码意义，帮助查找所需源码

Phrase02

先在终端中输入 objdump -d --no-show-raw-insn --disassemble=phase_02 ./bomb > phase_02.S, 获取 phrase_02 的汇编代码, 并让 dsh 逐句解释。这一题没有其他的函数调用, 因此不需要 dump 其他函数的源码, 解题信息收集完毕:

1. 函数初始化

```
402080: endbr64                  ; CET 标记,无数据效果
402084: push %rbp
402085: mov   %rsp,%rbp          ; rbp = 帧基址
402088: push %r13                ; 保存 callee-saved
40208a: mov   %rsi,%r13          ; r13 = challenge(第 2 个参数)
40208d: mov   $0x6,%esi          ; valid_u32_fields 的第 2 参 = 6
402092: push %r12
402094: mov   %rdi,%r12          ; r12 = input(第 1 个参数)
402097: push %rbx                ; 保存 callee-saved,后面当返回值暂存用
402098: sub   $0x28,%rsp         ; 开 40 字节局部区
```

关键: 40208a: mov %rsi,%r13 challenge 所在地址

由 40208d: mov $0x6,%esi 可知这一题的 count=6

402098: sub $0x28,%rsp 为这读取的 6 个数分配内存, 即后面的 v[i]

2. 格式校验

成功:

```
40209c: call 402c00 <valid_u32_fields>   ; 校验 input 是否恰好 6 个十进制 u32 字段
4020a1: test %al,%al                     ; 返回值是 bool,在 al
4020a3: jne   4020b8                     ; 通过 → 继续读入
```

关键: 4020a3: jne 4020b8 说明成功就跳过失败部分的代码, 开始读取 6 个整数

失败:

```
4020a5: xor   %ebx,%ebx          ; ebx = 0 → 将来返回 false
4020a7: lea   -0x18(%rbp),%rsp ; 把 rsp 拨回 rbx 的保存位置(见下文"为什么用 lea")
4020ab: mov   %ebx,%eax          ; 返回值 = ebx
4020ad: pop   %rbx / 4020ae: pop %r12 / 4020b0: pop %r13 / 4020b2: pop %rbp / 4020b3: ret
4020b4: nopl 0x0(%rax)           ; 对齐用 nop
```

3. 读入 6 个整数

```
4020b8: mov   %eax,%ebx          ; ebx = 1(valid_u32_fields 通过),作为最终返回值 true
4020ba: lea   -0x2c(%rbp),%rax ; rax = &v[5]
4020be: lea   -0x3c(%rbp),%rcx ; rcx = &v[1]
4020c2: mov   $0x403173,%esi     ; 格式串地址(.rodata,内容形如 "%u %u %u %u %u %u")
4020c7: push %rax                ; 第 8 个实参 = &v[5](栈上传参)
4020c8: lea   -0x30(%rbp),%rax ; rax = &v[4]
```

```
4020cc: lea    -0x40(%rbp),%rdx ; rdx = &v[0](第 3 个实参)
4020d0: mov    %r12,%rdi         ; rdi = input
4020d3: push %rax                ; 第 7 个实参 = &v[4](最后压栈,位于 rsp)
4020d4: lea    -0x34(%rbp),%r9   ; r9 = &v[3]
4020d8: lea    -0x38(%rbp),%r8   ; r8 = &v[2]
4020dc: xor    %eax,%eax         ; al = 0:变参调用约定(用到的 SSE 寄存器个数)
4020de: call 4011e0 <__isoc99_sscanf@plt>
4020e3: pop    %rdx              ; 清掉两个栈上传参
4020e4: pop    %rcx
4020e5: cmp    $0x6,%eax         ; 是否恰好转换出 6 项
4020e8: jne    4020a5            ; 否 → 失败
```

关键：8 个实参依次对应：rdi=输入串、esi=格式串、rdx=&v[0]、rcx=&v[1]、r8=&v[2]、r9=&v[3]、栈上第一个=&v[4]、栈上第二个=&v[5]。所以 v[0..5] 与输入顺序一一对应。

4. 第一个数

```
4020ea: mov    %r13,%rax         ; rax = challenge
4020ed: mov    -0x40(%rbp),%esi ; esi = v[0]
4020f0: shr    $0x14,%rax        ; challenge >> 20
4020f4: and    $0xf,%eax         ; 取低 4 位 → 0..15
4020f7: lea    (%rax,%rax,2),%eax ; eax = eax + eax*2 = 3 * 该值
4020fa: cmp    %eax,%esi         ; v[0] 与它比较
4020fc: jne    4020a5            ; 不等 → 失败
```

关键：lea (%rax,%rax,2) 的寻址式是 base + index*scale，此处 base 与 index 同为 %rax、scale=2，因此等价于乘 3

则 v[0] == 3 * ((challenge >> 20) & 0xf)

5. 循环

```
4020fe: mov    $0xa,%ecx         ; ecx = 10:第一次的移位量
402103: mov    $0x1,%edx         ; edx = 1:循环下标 i
402108: cmp    $0x6,%rdx         ; 循环头:i == 6 ?
40210c: je     4020a7            ; 是 → 走收尾,返回 ebx(=1,true)
40210e: mov    %r13,%rax         ; rax = challenge
402111: shr    %cl,%rax          ; challenge >> cl(cl 每轮 +5)
402114: add    $0x5,%ecx         ; 移位量 += 5
402117: and    $0x3f,%eax        ; 取低 6 位 → 0..63
40211a: xor    %edx,%eax         ; 再与下标 i 异或
40211c: lea    (%rax,%rsi,4),%eax ; eax = 上一个值*4 + 上面这段
40211f: mov    -0x40(%rbp,%rdx,4),%esi ; esi = v[i](本次读入的值)
402123: add    $0x1,%rdx         ; i++
402127: cmp    %eax,%esi         ; v[i] 与期望值比较
402129: je     402108            ; 相等 → 下一轮
40212b: jmp    4020a5            ; 不等 → 失败
```

关键：402103: mov $0x1,%edx 初始化循环的标量

402103: mov $0x1,%edx 限制循环次数

40210c: je 4020a7

402129: je 402108 这两句实现循环与跳出循环的功能

%esi 进循环前它是 v[0]；每轮 40211c 用它算“上一个值”，40211f 又把它改写成刚读入的 v[i]，下一轮正好继续当“上一个值”，所以循环不需要真的去读 v[i-1]

402111: shr %cl,%rax 是变量移位：ecx 从 10 开始、每轮 add $0x5，所以移位量依次是 10、15、20、25、30，下标 i 则是 1、2、3、4、5

402117: and $0x3f,%eax

40211a: xor %edx,%eax 先从 challenge 中取出 6 位，再与下标异或，得到与前一项无关的“增量部分”

40211c: lea (%rax,%rsi,4),%eax = rsi*4 + rax

6. 结束

4020a7: lea -0x18(%rbp),%rsp

4020ab: mov %ebx,%eax ；通过 = 1,失败 = 0(4020a5 里 xor 置的)

4020ad..4020b2: pop %rbx / %r12 / %r13 / %rbp

4020b3: ret

关键： 成功与失败两条路径共用一个出口，用 %ebx 传递布尔结

由上述分析，可得以下函数

```
int solve_phase_2(uint64_t challenge, char *out, size_t cap) {
    uint32_t values[MAX_FIELDS] = {0};
    size_t count = 6;
    values[0] = 3u * ((uint32_t)(challenge >> 20) & 0xfu);
    for (unsigned i = 1; i < 6; ++i) {
        uint32_t part = ((uint32_t)(challenge >> (10 + 5 * (i - 1))) & 0x3fu) ^ i;
        values[i] = values[i - 1] * 4u + part;
    }
    return write_answer(out, cap, values, count);
}
```

Seed:





Agents：帮助阅读汇编语言，解释源码

Phrase03

先在终端中输入 objdump -d --no-show-raw-insn --disassemble=phase_03 ./bomb > phase_03.S，获取 phrase_03 的汇编代码，并让 dsh 逐句解释。

Phase_03:

```
402240: endbr64
402244: push %rbp
402245: mov    %rsp,%rbp
402248: push %r12
40224a: mov    %rsi,%r12                ; r12 = challenge（第 2 个参数）
40224d: mov    $0x2,%esi                ; valid_u32_fields 的字段数 = 2
402252: push %rbx
402253: mov    %rdi,%rbx                ; rbx = input
402256: sub    $0x10,%rsp               ; 16 字节局部区：两个 u32
40225a: call 402c00 <valid_u32_fields>
40225f: test %al,%al
402261: jne    402270                   ; 通过 → 读入
402263: xor    %eax,%eax                ; 失败：返回 false
402265: add    $0x10,%rsp
402269: pop    %rbx / 40226a: pop %r12 / 40226c: pop %rbp / 40226d: ret
40226e: xchg %ax,%ax                    ; 对齐 nop
402270: xor    %eax,%eax                ; al = 0（sscanf 变参约定）
402272: lea    -0x14(%rbp),%rcx         ; 第 2 个输出 → v[1]
402276: lea    -0x18(%rbp),%rdx         ; 第 1 个输出 → v[0]
40227a: mov    $0x40317f,%esi           ; 格式串（"%u %u"）
40227f: mov    %rbx,%rdi                ; input
402282: call 4011e0 <__isoc99_sscanf@plt>
402287: cmp    $0x2,%eax
40228a: jne    402263                   ; 必须恰好 2 个数
```

由标黄源码可知答案是 2 个 u32 数字，v[0]存在 -0x18(%rbp)、v[1]存在 -0x14(%rbp)

```
40228c: mov    %r12d,%edi               ; edi = challenge 的低 32 位
40228f: xor    %eax,%eax                ; 先把 eax 清零（后面 sete 只写 al）
402291: and    $0x7,%edi                ; edi = challenge & 7  → 分派下标
402294: cmp    %edi,-0x18(%rbp)
402297: jne    402265                   ; v[0] 必须等于 (challenge & 7)
```

由标黄代码可知这个部分的代码限制了 v[0] = challenge & 7（范围在 0-7）并在后续调用 p3_dispatch 函数时会用到这个值

```
402299: mov    %r12,%rax
40229c: mov    %r12,%rsi
40229f: shr    $0x3,%rax                    ; rax = challenge >> 3
4022a3: shr    $0x20,%rsi                   ; rsi = challenge >> 32
4022a7: xor    %eax,%esi            ; esi = (uint32)(challenge>>32) ^ (uint32)(challenge>>3)
4022a9: call 4021e0 <p3_dispatch>   ; edi =  下标(已就绪),  esi = x
4022ae: cmp    %eax,-0x14(%rbp)
4022b1: sete %al                            ; v[1] ==  返回值
4022b4: jmp    402265                         ;  统一收尾
```

esi = (uint32_t)(challenge >> 32) ^ (uint32_t)(challenge >> 3)
之后 p3_dispatch 函数用了前面计算的 v[0]和这里计算呢出来的 esi
最后的结果为 v[1]并返回

分析完 phrase03 之后分析其中调用的函数 p3_dispatch：

```
4021e0: endbr64
4021e4: mov    %edi,%eax                ; eax =  下标
4021e6: mov    %esi,%edi                ; edi = x（转交给  case  的第  1  个参数）
4021e8: cmp    $0x7,%eax
4021eb: ja     402235                   ;  无符号大于  7  →  越界
4021ed: notrack jmp *0x4057e0(,%rax,8)    ;  跳转表，  8  字节一项
4021f5: nopl (%rax)                     ;  下面每  8  字节一个“落地垫”，  各自  jmp  到真正的  case
4021f8: jmp    4021b0 <p3_case_6>
402200: jmp    4021a0 <p3_case_5>
402208: jmp    402140 <p3_case_1>
402210: jmp    402130 <p3_case_0>
402218: jmp    402170 <p3_case_3>
402220: jmp    4021d0 <p3_case_7>
402228: jmp    402190 <p3_case_4>
402230: jmp    402150 <p3_case_2>
402235: xor    %eax,%eax
402237: ret                             ;  越界时返回  0
```

4021e4: mov   %edi,%eax 和 4021e6: mov   %esi,%edi 把外部的参数拿到函数里
下面的一系列 jmp 源码构成了 switch 的分支，对应到 8 个不同的函数
因此我也把这 8 个小函数以此 dump 出来转化为 c 语言函数

p3_case_0：

```
402134: lea 0xf7b0(%rdi,%rdi,2),%eax     ; rdi + rdi*2 + 0xf7b0
```

static uint32_t p3_case_0(uint32_t x) { return x * 3u + 0xf7b0u; }

p3_case_1:
402144: lea 0x2f049(%rdi,%rdi,4),%eax ; rdi*4 + rdi + 0x2f049

```
static uint32_t p3_case_1(uint32_t x) { return x * 5u + 0x2f049u; }
```

p3_case_2:
402154: mov %edi,%eax ; eax = x
402156: shr $0x2,%edi ; edi = x >> 2
402159: xor $0x14e1,%eax ; eax = x ^ 0x14e1
40215e: add %edi,%eax ; eax += x>>2

```
static uint32_t p3_case_2(uint32_t x) { return (x ^ 0x14e1u) + (x >> 2); }
```

p3_case_3:
402174: lea 0x0(,%rdi,4),%eax ; eax = x*4
40217b: xor $0xa2d0,%eax ; eax ^= 0xa2d0

```
static uint32_t p3_case_3(uint32_t x) { return (x * 4u) ^ 0xa2d0u; }
```

p3_case_4:
402194: mov %edi,%eax
402196: shr $1,%edi ; x >> 1
402198: xor $0x148f,%eax ; x ^ 0x148f
40219d: add %edi,%eax

```
static uint32_t p3_case_4(uint32_t x) { return (x ^ 0x148fu) + (x >> 1); }
```

p3_case_5:
4021a4: lea 0x4c68a(%rdi,%rdi,4),%eax

```
static uint32_t p3_case_5(uint32_t x) { return x * 5u + 0x4c68au; }
```

p3_case_6:
4021b4: lea 0x0(,%rdi,4),%eax ; 4x
4021bb: xor $0x1272,%eax

```
static uint32_t p3_case_6(uint32_t x) { return (x * 4u) ^ 0x1272u; }
```

p3_case_7:
4021d4: lea 0x9852(%rdi,%rdi,4),%eax

```
static uint32_t p3_case_7(uint32_t x) { return x * 5u + 0x9852u; }
```

因此最后我们得到了如下的函数:

```c
static uint32_t p3_case_0(uint32_t x) { return x * 3u + 0xf7b0u; }
static uint32_t p3_case_1(uint32_t x) { return x * 5u + 0x2f049u; }
static uint32_t p3_case_2(uint32_t x) { return (x ^ 0x14e1u) + (x >> 2); }
static uint32_t p3_case_3(uint32_t x) { return (x * 4u) ^ 0xa2d0u; }
static uint32_t p3_case_4(uint32_t x) { return (x ^ 0x148fu) + (x >> 1); }
static uint32_t p3_case_5(uint32_t x) { return x * 5u + 0x4c68au; }
static uint32_t p3_case_6(uint32_t x) { return (x * 4u) ^ 0x1272u; }
static uint32_t p3_case_7(uint32_t x) { return x * 5u + 0x9852u; }

static uint32_t p3_dispatch(uint32_t index, uint32_t x) {
    switch (index & 7u) {
        case 0:  return p3_case_1(x);
        case 1:  return p3_case_0(x);
        case 2:  return p3_case_3(x);
        case 3:  return p3_case_7(x);
        case 4:  return p3_case_4(x);
        case 5:  return p3_case_2(x);
        case 6:  return p3_case_6(x);
        default: return p3_case_5(x);
    }
}

int solve_phase_3(uint64_t challenge, char *out, size_t cap) {
    uint32_t values[MAX_FIELDS] = {0};
    size_t count = 2;
    uint32_t idx = (uint32_t)challenge & 7u;
    uint32_t x   = (uint32_t)(challenge >> 32)^ (uint32_t)(challenge >> 3);
    values[0] = idx;
    values[1] = p3_dispatch(idx, x)
    return write_answer(out, cap, values, count);
}
```

Seed:





Agents: 使用 dsh 帮助阅读源码和 dump 源码

Phrase04

先在终端中输入 objdump -d --no-show-raw-insn --disassemble=phase_04 ./bomb > phase_04.S, 获取 phrase_04 的汇编代码, 并让 dsh 逐句解释。其中发现有函数调用 p4_walk, 于是让 dsh 一起 dump 出来并解释。

phrase_04:

```
402330  endbr64                                  ; CET 间接跳转目标标记，无数据效果
402334  push %rbp                                ; 保存调用者的帧指针
402335  mov  %rsp,%rbp                           ; 建立本函数帧基址
402338  push %r12                                ; 保存 callee-saved 寄存器 r12
40233a  mov  %rsi,%r12                           ; r12 = challenge（第 2 个参数，函数全程要用）
40233d  mov  $0x1,%esi                           ; valid_u32_fields 的第 2 参 = 1, 口令只有 1 个字段
402342  push %rbx                                ; 保存 callee-saved 寄存器 rbx
402343  mov  %rdi,%rbx                           ; rbx = input（第 1 个参数），跨 sscanf 保存
402346  sub  $0x10,%rsp                         ; 开 16 字节局部区，-0x14(%rbp) 是那个 u32 变量
40234a  call 402c00 <valid_u32_fields>           ; 校验输入为恰好 1 个十进制 u32
40234f  test %al,%al                             ; 取返回的 bool（在 al 里）
402351  jne  402360                              ; 校验通过则跳去读入
402353  add  $0x10,%rsp                         ; 失败路径：归还局部区
402357  xor  %eax,%eax                           ; 返回值 = false
402359  pop  %rbx                                ; 恢复 rbx
40235a  pop  %r12                                ; 恢复 r12
40235c  pop  %rbp                                ; 恢复帧指针
40235d  ret                                      ; 返回 false
40235e  xchg %ax,%ax                             ; 2 字节 nop，代码对齐
402360  xor  %eax,%eax                           ; al = 0：sscanf 是变参函数，约定 al 放 SSE 寄存器个数
402362  lea  -0x14(%rbp),%rdx                    ; sscanf 第 3 个参数 = 输出变量地址
402366  mov  $0x403182,%esi
; 第 2 个参数 = .rodata 里的格式串（与第 1 关同址，即 "%u"）
40236b  mov  %rbx,%rdi                           ; 第 1 个参数 = 输入串
40236e  call 4011e0 <__isoc99_sscanf@plt>        ; 从文本解析一个 u32
402373  cmp  $0x1,%eax                           ; 是否恰好转换 1 项
402376  jne  402353                              ; 否则走失败收尾，返回 false
```

由上述源码块可知，challenge 保存在%r12，count=1，input 在%rbx

```
402378  mov  %r12,%rcx                          ; rcx = challenge（下面要 >>7）
40237b  mov  %r12,%rax                          ; rax = challenge（下面要 >>32）
40237e  mov  %r12d,%edi                         ; edi = challenge 的低 32 位
402381  xor  %esi,%esi                          ; esi = 0 → p4_walk 第 2 个实参 lo = 0
402383  shr  $0x20,%rax                         ; rax = challenge >> 32
402387  shr  $0x7,%rcx                          ; rcx = challenge >> 7
```

```
40238b    and    $0x7f,%edi                    ; edi = challenge & 127 → 第 1 个实参 key ∈ [0,127]
40238e    mov    $0x7f,%edx                    ; edx = 127 → 第 3 个实参 hi = 0x7f
402393    xor    %eax,%ecx                     ; ecx = (uint32)(c>>32) ^ (uint32)(c>>7) → 第 4 个实参 salt
402395    call 4022c0 <p4_walk>                ; 实参依次为 key, 0, 0x7f, salt
40239a    cmp    %eax,-0x14(%rbp)              ; 输入值与返回值作 32 位比较
40239d    sete %al                             ; 相等则 al = 1
```

此处分别准备 p4_walk 需要的四个参数，并调用 p4_walk

```
4023a0    add    $0x10,%rsp                    ; 成功路径：归还局部区
4023a4    pop    %rbx                          ; 恢复 rbx
4023a5    pop    %r12                          ; 恢复 r12
4023a7    pop    %rbp                          ; 恢复帧指针
4023a8    ret                                  ; 返回判定结果
```

成功后函数返回值

p4_walk:

```
4022c4: mov    %edi,%eax            ; eax = key
4022c6: mov    %edx,%edi            ; edi = hi
4022c8: sub    %esi,%edi            ; edi = hi - lo
4022ca: shr    $1,%edi              ; 无符号右移 1 位
4022cc: add    %esi,%edi            ; edi = lo + (hi-lo)/2    ← mid
4022ce: cmp    %eax,%edi
4022d0: je     402320               ; key == mid → 基本情况
```

按寄存器推断 p4_walk 函数形式：uint32_t p4_walk(uint32_t key /*edi*/, uint32_t lo /*esi*/, uint32_t hi /*edx*/, uint32_t salt /*ecx*/)
此处计算 mid 参数，mid = lo + (hi - lo) / 2，lo=0、hi=0x7f、key ∈ [0,127]，这是闭区间上的二分查找

```
402320: lea    0x7e(%rdi),%eax    ; eax = mid + 0x7e（此时 mid == key）
402323: xor    %ecx,%eax          ; eax ^= salt
402325: ret
```

计算(mid + 0x7e) ^ salt；也就是 (key + 0x7e) ^ salt

```
4022d2: push %rbp                 ; 建立栈帧（因为要递归调用）
4022d3: mov    %rsp,%rbp
4022d6: push %rbx                 ; rbx 是 callee-saved，得保存
4022d7: lea    (%rdi,%rcx,1),%ebx ; ebx = mid + salt（**先算好**，递归后还要用）
```

```
4022da: sub    $0x8,%rsp
4022de: cmp    %edi,%eax            ; key 与 mid 比较
4022e0: jb     402300               ; **无符号**小于 → 左半区
```

此处构建递归的框架。ebx 需要在递归调用前就算好，因为 %ebx 跨调用会被 callee 保存/恢复，而 %ecx（salt）虽然也跨调用有效，但 mid 是本帧的局部值，必须保存住

```
4022e2: lea    0x1(%rdi),%esi      ; lo = mid + 1
4022e5: mov    %eax,%edi             ; 第 1 参仍是 key
4022e7: call 4022c0 <p4_walk> ; hi、salt 不变
4022ec: lea    (%rax,%rax,2),%eax ; eax = 3 * 返回值
4022ef: xor    %ebx,%eax             ; eax ^= (mid + salt)
4022f1: mov    -0x8(%rbp),%rbx    ; 恢复 rbx
4022f5: leave                        ; rsp = rbp, pop rbp
4022f6: xor    $0x2,%eax             ; 再异或常数 2
4022f9: ret
```

当 key>mid 时，进入这一部分代码
递归 p4_walk(key, mid+1, hi, salt)，返回 ((3 * r) ^ (mid + salt)) ^ 2

```
402300: lea    -0x1(%rdi),%edx    ; hi = mid - 1
402303: mov    %eax,%edi             ; 第 1 参仍是 key
402305: call 4022c0 <p4_walk> ; lo、salt 不变
40230a: lea    (%rax,%rax,2),%eax ; 3 * 返回值
40230d: xor    %ebx,%eax             ; ^= (mid + salt)
40230f: mov    -0x8(%rbp),%rbx
402313: leave
402314: xor    $0x15,%eax            ; 再异或常数 0x15
402317: ret
```

当 key<mid 时，进入这一部分代码
递归 p4_walk(key, lo, mid-1, salt)，返回 ((3 * r) ^ (mid + salt)) ^ 0x15

此处两部分的函数递归不是尾递归，子调用返回后还要乘 3、异或本帧的 mid + salt、再异或分支常数，因此是从基本情况逐层回代

由上述分析，我们得到以下函数：

```c
static uint32_t p4_walk(uint32_t key, uint32_t lo, uint32_t hi, uint32_t salt) {
    uint32_t mid = lo + (hi - lo) / 2;
    if (key == mid) return (mid + 0x7eu) ^ salt;
    uint32_t t = mid + salt;
    if (key > mid) {
        uint32_t r = p4_walk(key, mid + 1u, hi, salt);
        return (r * 3u ^ t) ^ 0x2u;
    }
    uint32_t r = p4_walk(key, lo, mid - 1u, salt);
    return (r * 3u ^ t) ^ 0x15u;
}
```

```c
int solve_phase_4(uint64_t challenge, char *out, size_t cap) {
    uint32_t values[MAX_FIELDS] = {0};
    size_t count = 1;
    uint32_t salt = (uint32_t)(challenge >> 32) ^ (uint32_t)(challenge >> 7);
    values[0] = p4_walk((uint32_t)challenge & 0x7fu, 0u, 0x7fu, salt);
    return write_answer(out, cap, values, count);
}
```

Seed:





Agents: 使用 dsh 帮助阅读源码和 dump 函数

Phrase05

先在终端中输入 objdump -d --no-show-raw-insn --disassemble=phase_05 ./bomb > phase_05.S，获取 phrase_05 的汇编代码，并让 dsh 逐句解释。其中发现有函数调用 p5_traverse，于是让 dsh 一起 dump 出来并解释。

Phrase05：

```
402410   endbr64                                  ; CET 标记，无数据效果
402414   push %rbp                                ; 保存调用者帧指针
402415   mov   %rsp,%rbp                          ; 建立本函数帧基址
402418   push %r12                                ; 保存 callee-saved 寄存器 r12
40241a   mov   %rsi,%r12                          ; r12 = challenge（第 2 个参数）
40241d   mov   $0x2,%esi                          ; valid_u32_fields 的字段数 = 2
402422   push %rbx                                ; 保存 callee-saved 寄存器 rbx
402423   mov   %rdi,%rbx                          ; rbx = input（第 1 个参数）
402426   sub   $0x10,%rsp                         ; 开 16 字节局部区
40242a   call 402c00 <valid_u32_fields>           ; 校验输入为恰好 2 个十进制 u32
40242f   test %al,%al                             ; 取返回的 bool
402431   jne   402440                             ; 通过则继续
402433   xor   %eax,%eax                          ; 失败路径：返回 false
402435   add   $0x10,%rsp                         ; 归还局部区
402439   pop   %rbx                               ; 恢复 rbx
40243a   pop   %r12                               ; 恢复 r12
40243c   pop   %rbp                               ; 恢复帧指针
40243d   ret                                      ; 返回
40243e   xchg %ax,%ax                            ; 2 字节 nop，对齐
```

前期准备代码，challenge 存在 r12，count=2，输入不是两个参数则返回 false

```
402440   xor   %eax,%eax                          ; al = 0：sscanf 变参约定（SSE 寄存器个数）
402442   lea   -0x18(%rbp),%rcx                   ; 第 3 个输出指针 → v[1]（rbp-0x18）
402446   lea   -0x1c(%rbp),%rdx                   ; 第 2 个输出指针 → v[0]（rbp-0x1c）
40244a   mov   $0x40317f,%esi                     ; 格式串（与第 3 关同址，即 "%u %u"）
40244f   mov   %rbx,%rdi                          ; 输入串
402452   call 4011e0 <__isoc99_sscanf@plt>       ; 解析两个 u32
402457   cmp   $0x2,%eax                          ; 是否恰好转换 2 项
40245a   jne   402433                             ; 否则失败
```

把文本转成两个 u32，按顺序写入 v[0]（-0x1c）和 v[1]（-0x18），rdx 在前，对应第一个数

```
40245c   lea   -0x14(%rbp),%rsi                   ; p5_traverse 的第 2 个参数 = &out（rbp-0x14）
```

```
402460    mov    %r12,%rdi                    ; 第 1 个参数 = challenge
402463    call 4023b0 <p5_traverse>           ; 遍历链表
```

调用函数，同时提供一个指针让它写回“最后访问节点的 f1

```
402468    mov    %eax,%edx                    ; edx = 返回值（累加结果）
40246a    xor    %eax,%eax                    ; 先把返回值清零，供失败路径使用
40246c    cmp    %edx,-0x1c(%rbp)             ; v[0] 与累加结果比较
40246f    jne    402435                       ; 不等 → 直接走收尾，返回 0
402471    mov    -0x14(%rbp),%eax             ; eax = out（p5_traverse 写回的值）
402474    cmp    %eax,-0x18(%rbp)             ; v[1] 与 out 比较
402477    sete %al                            ; 相等则 al = 1
40247a    jmp    402435                       ; 统一收尾
```

检验条件，v[0] 等于累加结果，v[1] 等于写回的 out

```
402435    add    $0x10,%rsp
402439    pop    %rbx
40243a    pop    %r12
40243c    pop    %rbp
40243d    ret
```

公用结尾代码块，三条路径（校验失败、sscanf 失败、判定完成）共用，负责恢复栈和寄存器后返回，返回值在 eax

P5_traverse:

```
4023b4    mov    %rdi,%rax
4023b7    mov    %rdi,%rcx
4023ba    mov    %rsi,%r8                     ; r8 = out（本函数无调用，安全）
4023bd    and    $0x7,%eax                    ; idx  = challenge & 7
4023c0    shr    $0x3,%rcx                    ; rcx  = challenge >> 3（稍后取 &3 当步数）
```

rdi = challenge、rsi = out 指针，返回值 eax
从 challenge 里切出两组位，低 3 位用于选链表，接下来的 2 位定步数

```
4023c4    mov    0x405b20(,%rax,8),%rdx       ; rdx = p5_heads[idx]
```

p5_heads 是 8 项的指针表，每项 8 字节，按 idx 取出一条链表的头节点

```
4023cc    mov    %rdi,%rax
4023cf    shr    $0x20,%rdi                   ; rdi = challenge >> 32
4023d3    shr    $0x6,%rax                    ; rax = challenge >> 6
```

```
4023d7    xor    %edi,%eax                    ; x = (uint32)(c>>6) ^ (uint32)(c>>32)
```

再切两段 32 位，异或成累加器的初值来源，计算初值 x

```
4023d9    mov    0x4(%rdx),%esi               ; esi = node->f1
4023de    xor    $0xa140,%eax                 ; x ^= 0xa140
4023e3    lea    (%rax,%rax,2),%eax           ; *3
4023e6    add    (%rdx),%eax                  ; += node->f0
4023e8    mov    %esi,(%r8)                   ; *out = node->f1
4023eb    xor    %esi,%eax                    ; ^= node->f1
```

先对头节点做一次完整的计算：
acc = ((x ^ 0xa140) * 3 + node->f0) ^ node->f1
*out = node->f1
由 0x4(%rdx) 是 f1、(%rdx) 是 f0，可知节点结构是 {uint32 f0; uint32 f1; node *next;}（next 在偏移 8）

```
4023dc    mov    %ecx,%edi                    ; edi = (uint32)(c>>3)
4023ed    and    $0x3,%edi                    ; steps = (c>>3) & 3
4023f0    je     40240f                       ; steps == 0 → 直接返回
```

步数取 challenge 的第 34 位，范围 03。为 0 时只处理头节点就返回，否则进入循环再走 steps 个节点

```
4023f2    mov    $0x1,%ecx                    ; count = 1
4023f7    mov    0x8(%rdx),%rdx               ; node = node->next
4023fb    lea    (%rax,%rax,2),%eax           ; acc *= 3
4023fe    add    $0x1,%ecx                    ; count++
402401    mov    0x4(%rdx),%esi               ; esi = node->f1
402404    add    (%rdx),%eax                  ; acc += node->f0
402406    xor    %esi,%eax                    ; acc ^= node->f1
402408    mov    %esi,(%r8)                   ; *out = node->f1
40240b    cmp    %ecx,%edi                    ; steps 与 count 比较
40240d    jae    4023f7                       ; steps >= count → 继续
40240f    ret
```

每轮先沿 next 前进一个节点，再按同一个式子递推，并每轮覆盖 *out
acc = (acc * 3 + node->f0) ^ node->f1
*out = node->f1
最后返回

因为本关与结构体布局、指针、链表遍历有关，则可以先对数据表进行分析：

p5_heads（0x405b20，8 项 × 8 字节）依次为：
0x405bc0 0x405c00 0x405b70 0x405b90 0x405ba0 0x405bf0 0x405bd0
0x405c10

p5_nodes（0x405b60 起，每项 16 字节）共 12 个节点，字段是 f0、f1、next：
0x405b60: 0xef46, 0x352, NULL 0x405b70: 0x9973, 0x164, 0x405b90
0x405b80: 0x315f, 0xb61, 0x405b60 0x405b90: 0x1cfe, 0x924, 0x405ba0
0x405ba0: 0x6352, 0x731, 0x405bf0 0x405bb0: 0x1484, 0xa71, 0x405b80
0x405bc0: 0x3cda, 0x85d, 0x405c00 0x405bd0: 0x3be3, 0x53c, 0x405c10
0x405be0: 0x491e, 0x20a, 0x405bb0 0x405bf0: 0xe7a9, 0x426, 0x405bd0
0x405c00: 0x6893, 0x012, 0x405b70 0x405c10: 0xc0e9, 0x626, 0x405be0

连起来（每行只列前 4 个节点：
heads[0] = 0x405bc0 → 0x405c00 → 0x405b70 → 0x405b90
heads[1] = 0x405c00 → 0x405b70 → 0x405b90 → 0x405ba0
heads[2] = 0x405b70 → 0x405b90 → 0x405ba0 → 0x405bf0
heads[3] = 0x405b90 → 0x405ba0 → 0x405bf0 → 0x405bd0
heads[4] = 0x405ba0 → 0x405bf0 → 0x405bd0 → 0x405c10
heads[5] = 0x405bf0 → 0x405bd0 → 0x405c10 → 0x405be0
heads[6] = 0x405bd0 → 0x405c10 → 0x405be0 → 0x405bb0
heads[7] = 0x405c10 → 0x405be0 → 0x405bb0 → 0x405b80

因此对于整关来说，数据流是这样的：
phase_05 用 challenge & 7 选一条链表，用 (challenge >> 3) & 3 决定在这条链上走几个节点，用 (challenge>>6) ^ (challenge>>32) 作为初值，p5_traverse 从头节点开始，按 acc = (acc * 3 + f0) ^ f1 递推，每访问一个节点就把它的 f1 写进 out，最后 phase_05 要求输入的两个数分别等于：acc（累加结果）和 f1（最后访问节点的）

于是得到以下代码：

```
struct p5_node {
    uint32_t f0;
    uint32_t f1;
    int      next;
};

static const struct p5_node p5_nodes[12] = {
    { 0xef46u, 0x352u, -1 },
    { 0x9973u, 0x164u,  3 },
    { 0x315fu, 0xb61u,  0 },
    { 0x1cfeu, 0x924u,  4 },
    { 0x6352u, 0x731u,  9 },
    { 0x1484u, 0xa71u,  2 },
    { 0x3cdau, 0x85du, 10 },
    { 0x3be3u, 0x53cu, 11 },
    { 0x491eu, 0x20au,  5 },
    { 0xe7a9u, 0x426u,  7 },
    { 0x6893u, 0x012u,  1 },
    { 0xc0e9u, 0x626u,  8 },
};
```

```c
static const int p5_heads[8] = { 6, 10, 1, 3, 4, 9, 7, 11 };

static uint32_t p5_traverse(uint64_t challenge, uint32_t *last) {
    unsigned idx   = (unsigned)(challenge & 7u);
    unsigned steps = (unsigned)((challenge >> 3) & 3u);
    uint32_t x     = (uint32_t)(challenge >> 6) ^ (uint32_t)(challenge >> 32);

    int n = p5_heads[idx];
    uint32_t acc = (x ^ 0xa140u) * 3u + p5_nodes[n].f0;
    acc ^= p5_nodes[n].f1;
    *last = p5_nodes[n].f1;

    for (unsigned i = 0; i < steps; ++i) {
        n = p5_nodes[n].next;
        acc = acc * 3u + p5_nodes[n].f0;
        acc ^= p5_nodes[n].f1;
        *last = p5_nodes[n].f1;
    }
    return acc;
}
```

```c
int solve_phase_5(uint64_t challenge, char *out, size_t cap) {
    uint32_t values[MAX_FIELDS] = {0};
    size_t count = 2;
    uint32_t last = 0;
    values[0] = p5_traverse(challenge, &last);
    values[1] = last;
    return write_answer(out, cap, values, count);
}
```

Seed:





Agents：使用 dsh 分析源码并 dump 函数

Phrase06

先在终端中输入 objdump -d --no-show-raw-insn --disassemble=phase_06 ./bomb > phase_06.S，获取 phrase_06 的汇编代码，并让 dsh 逐句解释。其中发现有函数调用 p6_dispatch，于是让 dsh 一起 dump 出来并解释，发现有 6 个 op 函数指针来间接调用函数。

Phrase06：

```
402550   endbr64
402554   push %rbp
402555   mov    %rsp,%rbp
402558   push %r12
40255a   mov    %rsi,%r12              ; r12 = challenge
40255d   mov    $0x1,%esi              ；字段数 = 1
402562   push %rbx
402563   mov    %rdi,%rbx              ; rbx = input
402566   sub    $0x10,%rsp             ；局部区，-0x14(%rbp) 是唯一的 u32
```

保存 callee-saved 寄存器、缓存两个参数、开局部空间，其中 mov $0x1,%esi 说明 count=1

```
40256a   call 402c00 <valid_u32_fields>
40256f   test %al,%al
402571   jne    402580                 ；通过 → 读入
402573   add    $0x10,%rsp
402577   xor    %eax,%eax              ；返回 false
402579   pop    %rbx
40257a   pop    %r12
40257c   pop    %rbp
40257d   ret
40257e   xchg %ax,%ax                 ；对齐 nop
```

检验格式，不符合的返回 false

```
402580   xor    %eax,%eax              ; al = 0（sscanf 变参约定）
402582   lea    -0x14(%rbp),%rdx   ；输出指针
402586   mov    $0x403182,%esi      ；格式串 "%u"（与第 1、4 关同址）
40258b   mov    %rbx,%rdi              ; input
40258e   call 4011e0 <__isoc99_sscanf@plt>
402593   cmp    $0x1,%eax
402596   jne    402573                 ；不是恰好 1 个数 → 失败
```

把输入的字符串转化为整数 v[0]

```
402598    mov    %r12,%rdi              ; 实参 = challenge（整个 64 位）
40259b    call 402500 <p6_dispatch>
4025a0    cmp    %eax,-0x14(%rbp)     ; v[0] 与返回值比较
4025a3    sete %al                     ; 相等 → true
```

调用函数，判断 v[0] == p6_dispatch(challenge)

```
4025a6    add    $0x10,%rsp
4025aa    pop    %rbx
4025ab    pop    %r12
4025ad    pop    %rbp
4025ae    ret
```

P6_dispatch:

```
402504    push %rbp
402505    mov    %rsp,%rbp
402508    push %rbx
402509    mov    %rdi,%rbx              ; rbx = challenge（后面两次调用都要用）
40250c    shr    $0x6,%rdi             ; rdi = challenge >> 6
402510    mov    %rbx,%rax
402513    shr    $0x20,%rax            ; rax = challenge >> 32
402517    xor    %eax,%edi             ; edi = (uint32)(c>>6) ^ (uint32)(c>>32)
402519    mov    %rbx,%rax              ; rax = challenge（下面取低 3 位）
```

计算第一个参数 x = (uint32_t)(challenge >> 6) ^ (uint32_t)(challenge >> 32)

```
40251c    sub    $0x8,%rsp
402520    and    $0x38,%ebx            ; ebx = c & 0x38   ← 第二个下标的“字节偏移”
402523    and    $0x7,%eax             ; eax = c & 7        ← 第一个下标
```

计算两个表的下标
栈需要 sub $0x8 让 call 前满足 16 字节对齐（进入时 rsp ≡ 8，push rbp 后 ≡ 0，push rbx 后 ≡ 8，再减 8 才 ≡ 0）；这样 and $0x38 可以直接得到字节偏移

```
402526    call *0x405dc0(,%rax,8)     ; eax = p6_ops[challenge & 7](x)
```

第一次间接调用，按 challenge 的低 3 位选函数并调用

```
40252d    xor    $0x502d,%eax              ; 第一次结果 ^= 0x502d
402532    mov    %eax,%edi                  ; 作为第二次调用的实参
402534    call *0x405dc0(%rbx)          ; eax = p6_ops[(challenge>>3) & 7](上一步结果)
```

f2(f1(x) ^ 0x502d)，两次调用用的是同一张表，但下标取自 challenge 的不同三位

```
40253a    mov    -0x8(%rbp),%rbx
40253e    leave                          ; rsp = rbp；pop rbp
40253f    add    $0x42a,%eax             ；返回值 += 0x42a
402544    ret
```

恢复寄存器后把结果加上 0x42a 返回

因为这一关涉及到间接调用函数，所以把这 6 个 op 块列出来：

p6_op_0（0x402480）

```
402484    mov    %edi,%eax
402486    xor    $0x26df,%eax            ; eax = x ^ 0x26df
40248b    lea    (%rax,%rdi,2),%eax ; eax += 2x
```

f0(x) = (x ^ 0x26df) + 2x

p6_op_1（0x402490）

```
402494    mov    %edi,%eax
402496    ror    $0x2,%eax               ；循环右移 2 位
```

```
f1(x) = (x >> 2) | (x << 30)              /* 32 位循环右移 */
```

p6_op_2（0x4024a0）

```
4024a4    mov    %edi,%eax
4024a6    lea    (%rdi,%rdi,2),%edx ; edx = 3x
4024a9    shl    $0x5,%eax               ; eax = x << 5
4024ac    add    $0x9b1a,%eax            ; eax += 0x9b1a
4024b1    xor    %edx,%eax               ; eax ^= 3x
```

f2(x) = ((x << 5) + 0x9b1a) ^ (3x)

p6_op_3（0x4024c0）

```
4024c4    lea    0x2757(%rdi),%eax    ; eax = x + 0x2757
4024ca    shr    $0x3,%edi               ; edi = x >> 3
4024cd    xor    %edi,%eax               ; eax ^= x>>3
```

f3(x) = (x + 0x2757) ^ (x >> 3)

p6_op_4（0x4024d0）

```
4024d4    mov    %edi,%eax
4024d6    shr    $0x2,%eax              ; x >> 2
4024d9    xor    %edi,%eax              ; eax = (x>>2) ^ x
4024db    add    $0x2e14,%eax           ; eax += 0x2e14
```

f4(x) = ((x >> 2) ^ x) + 0x2e14

p6_op_5（0x4024f0）

```
4024f4    mov    %edi,%eax
4024f6    rol    $0x3,%eax              ; 循环左移 3 位
4024f9    xor    $0x2dac,%eax           ; eax ^= 0x2dac
```

f5(x) = ((x << 3) | (x >> 29)) ^ 0x2dac　　/* 32 位循环左移 */

由上述分析，可得以下代码:

```c
static uint32_t p6_op_0(uint32_t x) { return (x ^ 0x26dfu) + x * 2u; }
static uint32_t p6_op_1(uint32_t x) { return (x >> 2) | (x << 30); }
static uint32_t p6_op_2(uint32_t x) { return ((x << 5) + 0x9b1au) ^ (x * 3u); }
static uint32_t p6_op_3(uint32_t x) { return (x + 0x2757u) ^ (x >> 3); }
static uint32_t p6_op_4(uint32_t x) { return ((x >> 2) ^ x) + 0x2e14u; }
static uint32_t p6_op_5(uint32_t x) { return ((x << 3) | (x >> 29)) ^ 0x2dacu; }

typedef uint32_t (*p6_fn)(uint32_t);

static const p6_fn p6_ops[8] = {
    p6_op_4,
    p6_op_3,
    p6_op_3,
    p6_op_0,
    p6_op_5,
    p6_op_0,
    p6_op_2,
    p6_op_1,
};
```

```c
static uint32_t p6_dispatch(uint64_t challenge) {
    uint32_t x = (uint32_t)(challenge >> 6) ^ (uint32_t)(challenge >> 32);
    uint32_t first  = p6_ops[(unsigned)(challenge & 7u)](x);
    uint32_t second = p6_ops[(unsigned)((challenge >> 3) & 7u)](first ^ 0x502du);
    return second + 0x42au;
}

int solve_phase_6(uint64_t challenge, char *out, size_t cap) {
    uint32_t values[MAX_FIELDS] = {0};
    size_t count = 1;
    values[0] = p6_dispatch(challenge);
    return write_answer(out, cap, values, count);
}
```

Seed:





Agents：使用 dsh 帮助阅读源码和 dump 函数

Secret

我先借助 dsh dump 出来了所有相关的函数 secret_copy，secret_gate，secret_phase 和 secret_mix 并让 dsh 逐句分析源码

secret_copy（0x402690）：

```
402694   xor     %eax,%eax
4026a0   movzbl (%rsi,%rax,1),%edx     ; dl = src[i]（按字节读，零扩展）
4026a4   mov      %dl,(%rdi,%rax,1)     ; dst[i] = dl
4026a7   add      $0x1,%rax               ; i++
4026ab   test     %dl,%dl
4026ad   jne      4026a0                  ；没遇到 '\0' 就继续
4026af   ret
```

把 src（%rsi）逐字节拷到 dst（%rdi），**包含结尾的** '\0'，即 strcpy

secret_mix（0x4025b0）：

```
4025b4   mov     %edi,%eax          ; eax = x
4025b6   xor     $0xc249,%edi       ; edi = x ^ 0xc249
4025bc   shr     $0x3,%eax          ; eax = x >> 3
4025bf   xor     $0xbe4e,%eax       ; eax = (x>>3) ^ 0xbe4e
4025c4   add     %edi,%eax          ; eax += (x ^ 0xc249)
4025c6   ret
```

secret_mix(x) = ((x >> 3) ^ 0xbe4e) + (x ^ 0xc249)

secret_gate（0x4026b0）：

```
4026b4   push %rbp
4026b5   xor   %eax,%eax
4026b7   mov   $0x14,%ecx              ; 0x14 个 qword = 0xA0 字节
4026bc   mov   %rsp,%rbp
4026bf   push %r14
4026c1   mov   %rdx,%r14               ; r14 = 第 3 个参数（context）
4026c4   push %r13
4026c6   mov   %rsi,%r13               ; r13 = 第 2 个参数（challenge）
4026c9   push %r12
4026cb   lea   -0xd0(%rbp),%r12        ; r12 = 缓冲区首地址
4026d2   push %rbx
4026d3   mov   %rdi,%rbx               ; rbx = 第 1 个参数（输入行）
4026d6   mov   %r12,%rdi
4026d9   sub   $0xb0,%rsp
```

```
4026e0    rep stos %rax,(%rdi)        ; 从 rbp-0xd0 起清 0xA0 字节
4026e3    movl $0x0,(%rdi)              ; 再清 4 字节，合计 0xA4 = 164 字节
```

录入参数并建立缓冲区，缓冲区共 164 字节（rbp-0xd0 到 rbp-0x2d），先整体清零，避免残留数据

```
4026e9    mov    %rbx,%rdi              ; rdi = 输入行
4026ec    call 401190 <strlen@plt>
4026f1    sub    $0x24,%rax             ; rax = len - 36
4026f5    cmp    $0x7f,%rax
4026f9    jbe    402710                 ; 无符号 (len-36) <= 127 → 通过
4026fb    add    $0xb0,%rsp             ; 否则返回 0
402702    xor    %eax,%eax
402704    pop    %rbx                          ; 恢复各 callee-saved 寄存器
402705    pop    %r12
402707    pop    %r13
402709    pop    %r14
40270b    pop    %rbp
40270c    ret                                  ; 返回 0
40270d    nopl   (%rax)                       ; 4 字节 nop，用于对齐 402710
```

0x24 = 36、0x7f = 127，所以合法长度是 **36 ≤ len ≤ 163**，163 + 结尾 '\0' 正好等于 164，保证缓冲区不会溢出

```
402710    mov    %rbx,%rsi              ; 第 2 个参数 = 输入行（源）
402713    mov    %r12,%rdi              ; 第 1 个参数 = 缓冲区（目的）
402716    call   402690 <secret_copy>   ; strcpy(buf, input)，含结尾 '\0'
40271b    cmpl   $0x4e45504f,-0xb0(%rbp)  ; 比较 buf[32..35] 与 "OPEN"
402725    jne    4026fb                 ; 不是 "OPEN" → 走上面的失败收尾，返回 0
```

-0xb0(%rbp)为缓冲区首地址 + 0x20（32），0x4e45504f 按小端拆开是 4f 50 45 4e，即 ASCII 的 O、P、E、N，所以第 7 行的**第 32~35 个字符必须是** OPEN

```
402727    mov    %r14,%rdx              ; 第 3 个参数 = context
40272a    mov    %r13,%rsi              ; 第 2 个参数 = challenge
40272d    lea    -0xac(%rbp),%rdi       ; 第 1 个参数 = buf + 36
402734    call   4025d0 <secret_phase>  ; 解析并验证 OPEN 之后的 3 个数
402739    add    $0xb0,%rsp             ; 归还局部区
402740    pop    %rbx
402741    pop    %r12
402743    pop    %r13
402745    pop    %r14
402747    pop    %rbp
```

```
402748    ret                                    ; 返回值就是 secret_phase 的返回值
```

-0xac(%rbp) = 缓冲区 + 0x24 = +36。也就是说，OPEN 之后的第 37 个字符起是交给 secret_phase 解析的部分；secret_gate 直接把 secret_phase 的返回值作为自己的返回值

secret_phase（0x4025d0）：

```
4025d4    push %rbp
4025d5    mov    %rsp,%rbp
4025d8    push %r13
4025da    mov    %rdx,%r13              ; r13 = context
4025dd    push %r12
4025df    mov    %rsi,%r12              ; r12 = challenge
4025e2    mov    $0x3,%esi              ; 字段数 = 3
4025e7    push %rbx
4025e8    mov    %rdi,%rbx              ; rbx = tail（OPEN 之后的部分）
4025eb    sub    $0x18,%rsp
4025ef    call 402c00 <valid_u32_fields>
4025f4    test %al,%al
4025f6    jne    402610                 ; 通过 → 解析
4025f8    xor    %edx,%edx              ; 返回 0
4025fa    add    $0x18,%rsp
4025fe    mov    %edx,%eax
402600    pop    %rbx / 402601: pop %r12 / 402603: pop %r13 / 402605: pop %rbp / 402606:
ret
```

要求 OPEN 之后恰好是 3 个十进制 u32 字段

```
402610    xor    %eax,%eax              ; al = 0（sscanf 变参约定）
402612    lea    -0x24(%rbp),%r8        ; 第 5 个输出指针 → v2
402616    lea    -0x28(%rbp),%rcx       ; 第 4 个输出指针 → v1
40261a    mov    $0x40317c,%esi         ; 格式串
40261f    lea    -0x2c(%rbp),%rdx       ; 第 3 个输出指针 → v0
402623    mov    %rbx,%rdi
402626    call 4011e0 <__isoc99_sscanf@plt>
40262b    cmp    $0x3,%eax
40262e    jne    4025f8                 ; 必须恰好 3 个数
```

局部变量：v0 = -0x2c、v1 = -0x28、v2 = -0x24。
格式串地址 0x40317c 与前面几关的关系可以印证内容：0x403182 是 "%u"（1 个），0x40317f 是 "%u %u"（2 个），0x40317c 正好往前 3 个字节，就是 "%u %u %u"——链接器把这几份共享的字符串合并成了同一块

```
402630    mov    %r13d,%ecx              ; ecx = context 的低 32 位
402633    xor    %edx,%edx               ; edx = 0（后续失败路径的返回值）
402635    and    $0x3,%ecx               ; c = context & 3
402638    cmp    %ecx,-0x2c(%rbp)
40263b    jne    4025fa                  ; v0 必须等于 c
```

v0 = context & 3

```
40263f    mov    %r12,%rsi               ; rsi = challenge
402640    imul $0xc249,%ecx,%r9d     ; r9 = c * 0xc249
402647    mov    %r13,%r8                ; r8 = context
40264a    shr    $0x20,%rsi              ; rsi = challenge >> 32
40264e    shr    $0x20,%r8               ; r8  = context >> 32
402652    xor    %r12d,%esi              ; a = (uint32)challenge ^ (uint32)(challenge>>32)
402655    xor    %r13d,%r8d              ; b = (uint32)context    ^ (uint32)(context>>32)
402658    shr    $0x10,%r13              ; r13 = context >> 16
40265c    mov    %esi,%edi               ; edi = a
40265e    xor    %r13d,%r9d              ; r9 ^= (uint32)(context>>16)
402661    xor    %r8d,%edi               ; edi = a ^ b
402664    call 4025b0 <secret_mix>
402669    xor    %r9d,%eax               ; eax ^= r9
40266c    cmp    %eax,-0x28(%rbp)
40266f    jne    4025fa                  ; v1 必须等于它
```

a = (uint32_t)challenge ^ (uint32_t)(challenge>>32)
b = (uint32_t)context ^ (uint32_t)(context>>32)、c = context & 3
v1 = secret_mix(a ^ b) ^ ((c * 0xc249) ^ (uint32_t)(context >> 16))

```
402671    shr    $0x3,%esi               ; esi = a >> 3
402674    lea    0xbe4e(%r8,%rcx,1),%edx     ; edx = b + c + 0xbe4e
40267c    add    %esi,%edx               ; edx += (a >> 3)
40267e    xor    %edx,%eax               ; eax = v1 ^ edx
402680    cmp    -0x24(%rbp),%eax
402683    sete %dl                       ; 相等则 dl = 1
402686    jmp    4025fa
```

v2 = v1 ^ ((b + c + 0xbe4e + (a >> 3)) mod 2^32)
最后 jmp 4025fa → mov %edx,%eax 把 dl（0/1）变成返回值；前面的失败跳转都落在这里，此时 edx 已被 402633 清零，所以返回 0

综上，可得以下函数：

```c
static uint32_t secret_mix(uint32_t x) {
    return ((x >> 3) ^ 0xbe4eu) + (x ^ 0xc249u);
}

int solve_secret(uint64_t challenge, uint64_t context, char *out, size_t cap) {
    uint32_t a = (uint32_t)challenge ^ (uint32_t)(challenge >> 32);
    uint32_t b = (uint32_t)context   ^ (uint32_t)(context   >> 32);
    uint32_t c = (uint32_t)context & 3u;

    uint32_t v0 = c;
    uint32_t v1 = secret_mix(a ^ b) ^ ((c * 0xc249u) ^ (uint32_t)(context >> 16));
    uint32_t v2 = v1 ^ (b + c + 0xbe4eu + (a >> 3));

    static const char gate[] = "0123456789abcdefghijklmnopqrstuvOPEN";

    int size = snprintf(out, cap, "%s %" PRIu32 " %" PRIu32 " %" PRIu32,
                        gate, v0, v1, v2);
    if (size < 0 || (size_t)size >= cap) return 1;
    return 0;
}
```

Seed：

```
asus@DESKTOP-0TF26NC:~/Lab2-BombLab$ { ./defuser --seed 42; ./defuser --secret --seed 42; } | ./bomb --seed 42
```

```
asus@DESKTOP-0TF26NC:~/Lab2-BombLab$ { ./defuser --seed 42123; ./defuser --secret --seed 42123; } | ./bomb --seed 42123
```

```
Honor · 观测台旧备份

林夏准备关闭档案列表，忽然指着最下面一项：‘这里还有个备份。’
文件夹叫‘留给后来的人’，署名是 sky 和许遥。
扉页写着：‘交接完成后，如果你还有时间，可以到观测台坐坐。’
这份旧档案不参与城市的运行。你们沿着楼梯登上观测台，
桌上的阅读器仍在等待旧程序认可的输入。
城市已经安全了。是否继续读下去，由你们自己决定。

Secret phase defused.
[SAFE] 城市照常运行。附加档案已恢复。
```

Agents：使用 dsh 分析源码并 dump 函数